

รายงาน

การเทียบเคียงสมรรถนะ ระหว่าง AI-Testing เทียบกับ ขั้นตอนวิธี Symbolic Execution และ NSGA-II

Benchmarking AI-Testing vs Symbolic Execution and NSGA-II

โดย

673380271-3	นายชนภูมิ	แทนทุมมา
673380279-7	นายปองภพ	ศรีรักษ์
673380510-1	นางสาวจิรัชญา	เป้าจันทิก

อาจารย์ที่ปรึกษา

ผศ. ดร.ชิตสุธา สุ่มเล็ก

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353201 การประกันคุณภาพซอฟต์แวร์

ภาคเรียนที่ 1 ปีการศึกษา 2569

สาขาวิชาวิทยาการคอมพิวเตอร์ คณะ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

คำนำ

รายงานฉบับนี้จัดทำขึ้นเป็นส่วนหนึ่งของรายวิชา CP353201 การประกันคุณภาพซอฟต์แวร์ ภาคเรียนที่ 1 ปีการศึกษา 2569 โดยมีวัตถุประสงค์เพื่อศึกษาและเปรียบเทียบประสิทธิภาพของแนวทางการสร้างชุดทดสอบอัตโนมัติจำนวน 4 วิธี ได้แก่ NSGA-II, Symbolic Execution ผ่าน Symflower, ChatGPT และ GitHub Copilot บนชุดข้อมูล Defects4J ซึ่งเป็นชุดข้อมูลมาตรฐานที่ได้รับการยอมรับอย่างแพร่หลายในงานวิจัยและงานด้านการทดสอบซอฟต์แวร์

การดำเนินงานครอบคลุมการสร้างชุดทดสอบอัตโนมัติ การตรวจสอบความถูกต้องของชุดทดสอบ การวัดความสามารถในการตรวจพบข้อบกพร่อง (Fault Detection) การวัดความครอบคลุมของโค้ด (Code Coverage) รวมถึงการวิเคราะห์ข้อจำกัดและปัญหาที่พบระหว่างการทดลอง โดยใช้ข้อมูลจาก Defects4J จำนวน 17 โครงการ รวมทั้งสิ้น 854 Active Bugs เพื่อให้สามารถเปรียบเทียบผลลัพธ์ของแต่ละแนวทางภายใต้เงื่อนไขเดียวกันได้อย่างเป็นระบบ

คณะผู้จัดทำขอขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร.ชิตสุธา สุ่มเล็ก อาจารย์ผู้สอนและอาจารย์ที่ปรึกษารายวิชา ที่ได้ให้คำแนะนำ ข้อเสนอแนะ และแนวทางในการดำเนินโครงการตลอดระยะเวลาการศึกษา รวมถึงผู้พัฒนา Defects4J และเครื่องมือต่าง ๆ ที่ใช้ในการทดลอง ซึ่งมีส่วนสำคัญในการสนับสนุนการดำเนินงานและการประเมินผลในครั้งนี้

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจด้านการทดสอบซอฟต์แวร์อัตโนมัติ การสร้างชุดทดสอบด้วยเทคนิค Search-Based Testing, Symbolic Execution และ Generative AI ตลอดจนสามารถใช้เป็นข้อมูลอ้างอิงสำหรับการศึกษาและพัฒนางานในอนาคตได้ต่อไป หากมีข้อผิดพลาดหรือข้อบกพร่องประการใด คณะผู้จัดทำขอน้อมรับข้อเสนอแนะเพื่อใช้ในการปรับปรุงและพัฒนางานต่อไป

คณะผู้จัดทำ

สารบัญ

Contents

คำนำ	ก
สารบัญ	ข
สารบัญตาราง	จ
บทที่ 1 บทนำ	1
ที่มาและความสำคัญ	1
วัตถุประสงค์	1
ขอบเขตการศึกษา	2
บทที่ 2 เอกสารและทฤษฎีที่เกี่ยวข้อง	3
Symflower (Symbolic Execution)	3
Definition	3
Advantages	4
Symflower	6
NSGA-II	10
1. แนวคิดพื้นฐาน (Core Concept)	10
2. คำศัพท์เฉพาะ (Terminology)	10
3. หลักการทำงานหลัก (Key Concepts)	10
บทที่ 3 วิธีดำเนินงาน	12
3.1 ภาพรวมการดำเนินงาน	12
3.2 ชุดข้อมูลที่ใช้ในการทดลอง	12
3.3 เครื่องมือและสภาพแวดล้อม	13
3.3.1 เครื่องมือหลัก	13

3.3.2 การกำหนดค่าของแต่ละวิธี	14
3.4 ขั้นตอนการทดลองร่วม	14
3.5 การสร้างชุดทดสอบด้วย NSGA-II	15
3.6 การสร้างชุดทดสอบด้วย Symbolic Execution ผ่าน Symflower.....	15
3.7 การสร้างชุดทดสอบด้วย ChatGPT และ GitHub Copilot.....	15
3.8 เกณฑ์การประเมินผล.....	16
3.8.1 ความพร้อมในการประเมินชุดทดสอบ.....	16
3.8.2 การตรวจพบข้อบกพร่อง.....	17
3.8.4 จำนวนการทดสอบ	17
3.8.5 เวลาและทรัพยากร	17
3.9 การจัดการข้อผิดพลาดและการประเมินซ้ำ.....	18
3.10 การจัดเก็บผลการทดลอง	18
บทที่ 4 ผลการดำเนินงาน	19
4.1 ภาพรวมผลการทดลอง.....	19
4.2 ผลการทดลองด้วย NSGA-II	19
4.2.1 ผลตามสถานะ.....	19
4.2.2 ผลแยกตามโครงการ	20
4.2.3 ผลการรันซ้ำ.....	21
4.3 ผลการทดลองด้วย Symflower.....	21
4.3.1 ผลตามขั้นตอน.....	21
4.3.2 ผลแยกตามโครงการ	21
4.3.3 การทดลองเพิ่มเติมและการตรวจสอบผล.....	22

4.4 ผลการทดลองด้วย ChatGPT	23
4.4.1 ผลตามสถานะ	23
4.5 ผลการทดลองด้วย GitHub Copilot	25
4.5.1 ผลตามสถานะ	25
4.5.2 ผลแยกตามโครงการ	25
4.6 ผลเปรียบเทียบทั้ง 4 วิธี	26
4.6.1 การตรวจพบข้อบกพร่อง	26
4.7 ข้อมูลเฉพาะวิธีและผลบนกรณีร่วม	27
4.7.1 การใช้ token	27
4.7.2 การเปรียบเทียบบนกรณีร่วม	28
4.8 กรณีศึกษาและการตรวจสอบหลักฐาน	28
บทที่ 5 อภิปรายผล และสรุปผล	29
5.1 อภิปรายความสำเร็จของกระบวนการ	29
5.2 อภิปรายการตรวจพบข้อบกพร่องและ test oracle	29
5.3 อภิปรายความครอบคลุมของโค้ด	29
5.4 อภิปรายเวลาและทรัพยากร	30
5.5 ปัญหาและสิ่งที่เรียนรู้	30
5.6 ข้อจำกัดของการศึกษา	30
5.7 สรุปผลการศึกษา	31

สารบัญตาราง

ตารางที่ 1	ชุดข้อมูลที่ใช้ในการทดลอง.....	12
ตารางที่ 2	ตารางเครื่องมือหลัก.....	13
ตารางที่ 3	ตารางกำหนดค่าของแต่ละวิธี.....	14
ตารางที่ 4	ตารางรายละเอียดแต่ละ Prompt.....	16
ตารางที่ 5	ตารางรายละเอียดความหมายของสถานะที่ใช้รายงาน.....	16
ตารางที่ 6	ตารางการตรวจพบข้อบกพร่อง.....	17
ตารางที่ 7	ตารางรายละเอียดตำแหน่งของการจัดเก็บผลการทดลอง.....	18
ตารางที่ 8	ตารางภาพรวมผลการทดลอง.....	19
ตารางที่ 9	ตารางผลการทดลองด้วย NSGA-II ข้อมูลตามสถานะ (1).....	19
ตารางที่ 10	ตารางผลการทดลองด้วย NSGA-II ข้อมูลตามสถานะ (2).....	20
ตารางที่ 11	ตารางผลการทดลองด้วย NSGA-II ข้อมูลตามโครงการ.....	20
ตารางที่ 12	ตารางผลการทดลองด้วย Symflower ข้อมูลตามขั้นตอน.....	21
ตารางที่ 13	ตารางผลการทดลองด้วย Symflower ข้อมูลตามโครงการ.....	21
ตารางที่ 14	ตารางผลการทดลองด้วย ChatGPT ข้อมูลตามสถานะ.....	23
ตารางที่ 15	ตารางผลการทดลองด้วย ChatGPT ข้อมูลตามโครงการ.....	24
ตารางที่ 16	ตารางผลการทดลองด้วย GitHub Copilot ข้อมูลตามสถานะ.....	25
ตารางที่ 17	ตารางผลการทดลองด้วย GitHub Copilot ข้อมูลตามโครงการ.....	25
ตารางที่ 18	ตารางผลเปรียบเทียบทั้ง 4 วิธี ข้อมูลตามการตรวจพบข้อบกพร่อง.....	26
ตารางที่ 19	ตารางผลเปรียบเทียบทั้ง 4 วิธี ข้อมูลตามความครอบคลุมของโค้ด.....	26
ตารางที่ 20	ตารางผลเปรียบเทียบทั้ง 4 วิธี ข้อมูลตามเวลา.....	27
ตารางที่ 21	ตารางการใช้ token.....	27
ตารางที่ 22	ตารางเปรียบเทียบบนกรณีร่วม.....	28
ตารางที่ 23	ตารางรายละเอียดกรณีศึกษาที่พบ.....	28
ตารางที่ 24	ตารางรายละเอียดข้อจำกัดที่พบ.....	30

บทที่ 1

บทนำ

ที่มาและความสำคัญ

การทดสอบซอฟต์แวร์เป็นกระบวนการสำคัญในการประกันคุณภาพซอฟต์แวร์ โดยการเขียนชุดทดสอบด้วยตนเองต้องอาศัยความเข้าใจในโครงสร้างและพฤติกรรมของโปรแกรม รวมถึงต้องใช้เวลาและทรัพยากรของผู้พัฒนาเป็นจำนวนมาก จึงเกิดแนวทางการสร้างชุดทดสอบอัตโนมัติขึ้นเพื่อช่วยลดภาระดังกล่าว แนวทางการสร้างชุดทดสอบอัตโนมัติสามารถแบ่งได้หลายรูปแบบ เช่น Search-Based Software Testing, Symbolic Execution และ Generative AI โดยแต่ละแนวทางมีหลักการทำงานและข้อจำกัดแตกต่างกัน เช่น Search-based สามารถค้นหาชุดทดสอบที่เหมาะสมตามวัตถุประสงค์ที่กำหนด Symbolic Execution สามารถวิเคราะห์เส้นทางการทำงานของโปรแกรมและสร้างอินพุตเพื่อเข้าถึงเส้นทางต่าง ๆ ขณะที่ Generative AI สามารถใช้ข้อมูลบริบทและข้อกำหนดของโปรแกรมเพื่อสร้างชุดทดสอบพร้อม Assertion ได้ อย่างไรก็ตาม ประสิทธิภาพของแนวทางเหล่านี้ไม่ได้ขึ้นอยู่กับความสามารถในการสร้าง Test Case เพียงอย่างเดียว แต่ยังขึ้นอยู่กับข้อมูลที่ใช้เป็นบริบทในการสร้างชุดทดสอบและวิธีการกำหนด Test Oracle ด้วย ดังนั้นการเปรียบเทียบผลการทดลองจึงควรพิจารณาทั้งความสามารถในการสร้างชุดทดสอบ ความถูกต้อง ความครอบคลุมของโค้ด ความสามารถในการตรวจพบข้อบกพร่อง ตลอดจนขนาดของชุดทดสอบและต้นทุนในการสร้าง

วัตถุประสงค์

เพื่อสร้างชุดทดสอบอัตโนมัติสำหรับโปรแกรมภาษา Java ด้วยวิธี NSGA-II ผ่าน EvoSuite, Symbolic Execution ผ่าน Symflower, ChatGPT และ GitHub Copilot บนข้อบกพร่องจาก Defects4J จำนวน 854 กรณี ครอบคลุม 17 โครงการ ภายใต้กระบวนการทดลองเดียวกัน

เพื่อเปรียบเทียบสมรรถนะของแต่ละวิธีในด้านความพร้อมในการประเมินชุดทดสอบ ความสามารถในการตรวจพบ ข้อบกพร่อง ความครอบคลุมของโค้ด และเวลาและทรัพยากรที่ใช้

ขอบเขตการศึกษา

โครงการนี้จึงศึกษาการสร้างชุดทดสอบด้วย 4 แนวทาง ได้แก่ NSGA-II, Symbolic Execution ผ่าน Symflower, ChatGPT และ GitHub Copilot โดยใช้ Defects4J ซึ่งเป็นชุดข้อมูลข้อบกพร่องของโปรแกรมภาษา Java จำนวน 17 projects และ 854 active bugs เป็นฐานในการทดลอง

การศึกษานี้มุ่งเน้นการวิเคราะห์ผลของแต่ละแนวทางภายใต้กระบวนการทดลองที่กำหนด รวมถึงวิเคราะห์ข้อจำกัดที่เกิดขึ้นในแต่ละวิธี โดยเฉพาะผลของ Test Oracle ต่อความสามารถในการตรวจพบข้อบกพร่อง เพื่อให้เห็นลักษณะ จุดแข็ง และข้อจำกัดของแต่ละแนวทางอย่างชัดเจน

บทที่ 2

เอกสารและทฤษฎีที่เกี่ยวข้อง

Symflower (Symbolic Execution)

Definition

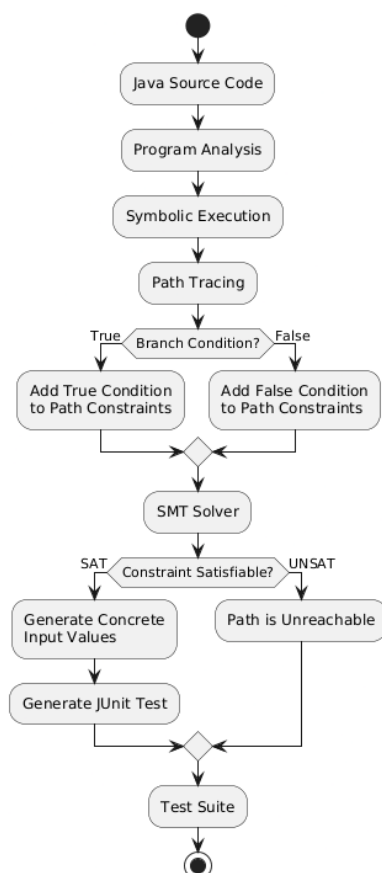
Symflower เป็นเครื่องมือสำหรับ Automated Test Generation ที่ใช้การวิเคราะห์โปรแกรมและการประมวลผลเชิงสัญลักษณ์ (Symbolic Execution) เพื่อวิเคราะห์เส้นทางการทำงานของโปรแกรม และสร้าง Unit Test

จากค่าที่สามารถทำให้โปรแกรมเดินไปตามเส้นทางต่าง ๆ ได้โดยอัตโนมัติ

ในกระบวนการทำงาน Symflower สามารถแปลงตรรกะของโปรแกรมเป็นเงื่อนไขทางคณิตศาสตร์ จากนั้นใช้ SMT Solver เพื่อหาค่า Input ที่ทำให้เงื่อนไขดังกล่าวเป็นจริง และนำค่าที่ได้ไปสร้างเป็น Unit Test ที่สามารถนำไปใช้งานจริงได้

Architecture / Workflow

Symflower Symbolic Execution and Test Generation



ขั้นตอนที่ 1: Java Source Code

Symflower รับ Source Code ของโปรแกรมหรือ Method ที่ต้องการทดสอบเป็นข้อมูลนำเข้า

ขั้นตอนที่ 2: Program Analysis

ระบบวิเคราะห์โครงสร้างและตรรกะของโปรแกรม เพื่อระบุคำสั่ง เงื่อนไข และเส้นทางการทำงานที่เกี่ยวข้องกับการทดสอบ

ขั้นตอนที่ 3: Symbolic Execution

ตัวแปรนำเข้าจะถูกแทนด้วย Symbolic Values แทนการใช้ค่าจริง ทำให้ระบบสามารถวิเคราะห์เงื่อนไขต่าง ๆ ของโปรแกรมได้

ขั้นตอนที่ 4: Path Tracing และ Path Splitting

เมื่อพบคำสั่งเงื่อนไข เช่น if-else ระบบจะแยกเส้นทางการทำงานออกเป็นหลาย Path และติดตามเงื่อนไขของแต่ละเส้นทาง

ขั้นตอนที่ 5: Path Constraints

เงื่อนไขที่พบในแต่ละเส้นทางจะถูกสะสมเป็นสมการหรือข้อจำกัดทางคณิตศาสตร์

ขั้นตอนที่ 6: SMT Solver

ข้อจำกัดของเส้นทางจะถูกส่งไปยัง SMT Solver เพื่อคำนวณหาค่า Input ที่สามารถทำให้เส้นทางนั้นเกิดขึ้นได้

ขั้นตอนที่ 7: Concrete Input Values

เมื่อได้คำตอบจาก Solver ระบบจะนำค่าทางสัญลักษณ์มาแปลงเป็นค่าจริง เช่น $x = 22$ และ $y = 11$

ขั้นตอนที่ 8: Unit Test Generation

ค่าที่คำนวณได้จะถูกนำไปใช้สร้าง Unit Test ที่สามารถรันได้จริง และรวม Test Case เหล่านี้เป็น Test Suite

กระบวนการดังกล่าวสอดคล้องกับตัวอย่างการทำงานของ Symflower ในไฟล์ที่ศึกษา ซึ่งประกอบด้วย Symbolic Initialization, Path Tracing, Path Constraint Construction, SMT Solving และ Concrete Value Generation

Advantages

จากแนวทางการทำงานของ Symflower สามารถสรุปข้อดีได้ดังนี้

1. สามารถสร้าง Test Case ได้แบบอัตโนมัติ

ช่วยลดความจำเป็นในการเขียน Unit Test ด้วยตนเอง และสามารถสร้าง Test Suite จากการวิเคราะห์โปรแกรมได้

2. สามารถค้นหา Input ที่เจาะจงต่อเส้นทางการทำงาน

Symbolic Execution ช่วยให้ระบบสร้างค่า Input ที่มุ่งไปยัง Execution Path ที่ต้องการ แทนที่จะอาศัยการสุ่มเพียงอย่างเดียว

3. มีแนวโน้มที่จะสร้าง Test Suite ที่กระชับ

การสร้าง Test จาก Path ที่แตกต่างกันช่วยลดการสร้าง Test Case ที่ซ้ำซ้อนกัน

4. สามารถตรวจจับข้อผิดพลาดที่เข้าถึงได้ยาก

เช่น Division by Zero, Null Pointer หรือข้อผิดพลาดที่ต้องใช้เงื่อนไขเฉพาะในการเข้าถึง

5. ผลลัพธ์มีความสามารถในการทำซ้ำได้

การคำนวณเชิงสัญลักษณ์เป็นกระบวนการที่อาศัยเงื่อนไขทางคณิตศาสตร์ จึงไม่ได้พึ่งพาการสุ่มในลักษณะเดียวกับ Random Testing

6. สามารถทำงานร่วมกับ AI ได้

Symflower มีแนวทางในการใช้ Smart Test Templates, ข้อมูลโครงสร้างของโปรเจกต์ และการประมวลผลภายหลังเพื่อช่วยให้ LLM สามารถสร้าง Test Code ได้เหมาะสมกับโปรเจกต์มากขึ้น

ครอบคลุมการทดสอบสูงสุด (Optimal Coverage):

เนื่องจากใช้วิธีคำนวณทางคณิตศาสตร์เพื่อหาค่าอินพุตที่เจาะจงไปยังทุกเส้นทาง

มันจึงรับประกันได้ว่าจะสามารถทดสอบโค้ดได้ครอบคลุมทุกกิ่งก้าน (Branch coverage) โดยไม่มีการเดาสุ่ม

ชุดทดสอบมีขนาดกะทัดรัด (Slim Test Suites): ต่างจากการทดสอบแบบสุ่ม (Fuzzing) ที่มักจะรันซ้ำซ้อนในเส้นทางเดิม

Symbolic Execution จะสร้างเคสทดสอบเพียง 1 เคสต่อ 1 เส้นทาง เท่านั้น ทำให้ไม่มีเคสทดสอบที่ซ้ำซ้อนและเข้าใจง่าย

ค้นหาคำบั๊กที่ซ่อนอยู่ลึกๆ ได้ดี (Deep Bug Detection): สามารถตรวจพบข้อผิดพลาดที่เกิดขึ้นเฉพาะในเงื่อนไขที่ซับซ้อนมากๆ

ได้อย่างแม่นยำ เช่น การหารด้วยศูนย์ (Division by zero), Pointer เป็นค่าว่าง (Null-pointer), หรือหน่วยความจำล้น (Buffer overflow) ซึ่งการสุ่มทั่วไปหาได้ยากมาก

ผลลัพธ์แน่นอนและทำซ้ำได้

(Deterministic): การรันโปรแกรมเชิงสัญลักษณ์จะให้ผลลัพธ์และค่าทดสอบเดิมเสมอทุกครั้งที่ทำงานกับโค้ดชุดเดิม

ไม่เหมือนการสุ่มที่ต้องพึ่งพาดวง

Limitations

แม้ Symflower จะสามารถสร้าง Unit Test อัตโนมัติได้ แต่ยังมีข้อจำกัดที่ควรพิจารณา

1. Path Explosion

เมื่อโปรแกรมมีเงื่อนไขและ Loop จำนวนมาก จำนวน Execution Paths อาจเพิ่มขึ้นอย่างรวดเร็ว

ส่งผลให้ต้องใช้เวลาและทรัพยากรในการวิเคราะห์มากขึ้น

2. External Environment

การทำงานกับ Database, Network หรือระบบภายนอกอาจทำให้การวิเคราะห์ด้วย Symbolic Execution มีความซับซ้อน และอาจต้องใช้ Stub หรือ Mock เพื่อจำลองสภาพแวดล้อมดังกล่าว

3. Loops และ Recursion

Loop ที่มีจำนวนรอบไม่จำกัดหรือ Recursive Call ที่ซับซ้อนอาจทำให้การสำรวจ Path ทำได้ยาก

4. Complex Data Structures

โครงสร้างข้อมูลที่มีความซับซ้อน เช่น Linked List หรือ Tree

อาจเพิ่มความยากในการวิเคราะห์สถานะของข้อมูลในหน่วยความจำ

5. External Dependencies

ไลบรารีหรือ Dependency ภายนอกบางส่วนอาจไม่สามารถถูกวิเคราะห์ได้อย่างสมบูรณ์ ทำให้การสร้าง Test Suite ในบางกรณีอาจไม่สมบูรณ์

ดังนั้น การมี Test Coverage สูงไม่ได้หมายความว่า Test Suite จะสามารถตรวจพบ Defect ได้ทุกประเภท

ในโครงการนี้จึงควรวัดทั้ง Code/Test Coverage และ Fault Detection Rate แยกจากกันตามเกณฑ์ของการทดลอง

Symflower

หลักการทำงานของ Symflower เพื่อให้เข้าใจง่ายที่สุด Symflower ไม่ได้ทำงานด้วยการสุ่มเดาเพื่อทดสอบระบบ แต่ใช้วิธีคำนวณตามหลักคณิตศาสตร์และการทำงานร่วมกับ AI ดังนี้:

การประมวลผลเชิงสัญลักษณ์ (Symbolic Execution) — หัวใจหลัก: แทนที่จะรันโปรแกรมด้วย "ค่าจริง" (เช่น ป้อนตัวเลข 5 หรือ 10 เข้าไปรันตรงๆ) Symflower จะป้อน "ค่าสัญลักษณ์" (เช่น $a0, b0$)

ซึ่งเปรียบเสมือนตัวแปรสมมติทางคณิตศาสตร์เข้าไปในระบบ จากนั้นระบบจะแกะรอยเส้นทางการทำงานของโค้ดในทุกๆ ทางแยกที่เป็นไปได้ (เช่น คำสั่งเงื่อนไข if-else) และสะสมเงื่อนไขเหล่านั้นไว้ในรูปแบบของระบบสมการคณิตศาสตร์ เรียกว่า เงื่อนไขเส้นทาง (Path Constraints) การแปลงโค้ดให้อยู่ในรูป Static Single Assignment (SSA):

ในการคำนวณทางคณิตศาสตร์ ตัวแปรจะต้องไม่มีการเปลี่ยนค่าซ้ำๆ Symflower จึงแปลงโค้ดให้อยู่ในรูปแบบ SSA

ซึ่งตัวแปรทุกตัวจะถูกตั้งชื่อเวอร์ชันย่อยกำกับไว้ (เช่น $x0, x1$) เพื่อให้ตัวแปรแต่ละเวอร์ชันมีค่าได้เพียงครั้งเดียวในแต่ละเส้นทาง

การแก้สมการด้วย SMT Solver: เงื่อนไขเส้นทางที่ถูกสะสมไว้จะถูกส่งต่อไปยังตัวแก้สมการคณิตศาสตร์ เช่น Z3 ของ Microsoft เพื่อทำการคำนวณหาค่าอินพุตจริงที่จะทำให้เงื่อนไขนั้นทำงานและเข้าถึงทุกเส้นทางได้
ค่าที่คำนวณได้นี้จะถูกนำมาเขียนเป็นชุดทดสอบ (Unit test) ที่รันได้จริงโดยอัตโนมัติ

การทำงานร่วมกับ AI (Generative AI & LLMs): การสร้างเทมเพลตทดสอบอัจฉริยะ (Smart Test Templates): Symflower จะสร้างโครงร่างชุดทดสอบ (Template) ที่ถูกต้องตามหลักไวยากรณ์ (Syntax) ทั้งคำสั่งนำเข้า (Imports), โครงสร้างคลาส และการจำลองออบเจกต์ภายนอก (Mockito Mocking) เพื่อให้ AI นำไปเขียนต่อได้โดยไม่เกิดข้อผิดพลาดด้าน Syntax

การจับคู่บริบทที่ถูกต้อง (RAG): Symflower จะทำการดึงข้อมูลโครงสร้างโปรเจกต์ (Symbol Map) ผ่านคำสั่ง symflower symbols เพื่อส่งให้ AI ทำให้ AI ทราบบริบทที่ถูกต้องของระบบ และป้องกันไม่ให้ AI เกิดอาการ "หลอน" (Hallucination)
การซ่อมแซมโค้ดอัตโนมัติ (Deterministic Post-Processing): เมื่อ AI เขียนโค้ดเสร็จ Symflower จะใช้คำสั่ง symflower fix เพื่อแก้ข้อผิดพลาดที่มักเกิดจาก AI เช่น สัมผัสประกาศแพ็คเกจ หรือลิงก์นำเข้าโค้ดพัง ช่วยเพิ่มประสิทธิภาพการรันโค้ดของ AI ได้สูงขึ้นถึง 26% โดยเฉลี่ย

ข้อดีของ Symflower

สร้างชุดทดสอบที่สั้น กระชับ และครอบคลุม (Slim and High-Coverage Test Suites): ต่างจากการทดสอบแบบเดาสุ่ม (Fuzzing) ที่อาจได้ผลลัพธ์ซ้ำซ้อน Symflower จะคำนวณทางคณิตศาสตร์เพื่อสร้างชุดทดสอบขึ้นมาเพียง 1 เคส ต่อ 1 เส้นทางการทำงานเท่านั้น ทำให้ได้ชุดทดสอบที่กระชับ ไม่ซ้ำซ้อน และครอบคลุมทุกกิ่งก้านของโค้ด
ให้ผลลัพธ์ที่แน่นอน (Deterministic): การคำนวณแบบคณิตศาสตร์ทำให้มั่นใจได้ว่า ไม่ว่าจะรันกี่ครั้ง บนโค้ดเดิม ค่าอินพุตและผลลัพธ์ชุดทดสอบที่ได้จะออกมาเหมือนเดิมเสมอ ไม่มีการสุ่มเดา
ตรวจจับบั๊กและข้อผิดพลาดได้เรียลไทม์ (Test-backed Diagnostics): หากโค้ดที่เขียนมีโอการันแล้วพัง (เช่น การหารด้วยศูนย์, Pointer เป็นค่าว่าง, หรือ Array เกินขอบเขต) ระบบจะไฮไลต์จุดเสี่ยงเหล่านั้นในหน้าเขียนโค้ดทันที
พร้อมป้อนเคสทดสอบที่สามารถนำไปรันเพื่อจำลองและแก้ไขปัญหาได้ทันที
จำลองออบเจกต์ภายนอกให้อัตโนมัติ (Automatic Mocking): รองรับการสร้าง Mock และตรวจสอบพฤติกรรมผ่าน Mockito สำหรับ Java และ Spring Boot โดยอัตโนมัติ
ทำงานแบบ Local 100% ปลอดภัยต่อทรัพย์สินทางปัญญา:
โค้ดของบริษัทหรือหน่วยงานจะไม่ถูกส่งออกไปภายนอกเครื่องคอมพิวเตอร์ของคุณในกระบวนการประมวลผลเชิงสัญลักษณ์ (Symbolic Execution) ทำให้ไม่มีความเสี่ยงเรื่องข้อมูลรั่วไหล

ข้อเสีย / ข้อจำกัดของ Symflower

รองรับภาษาจำกัดในโมเดลคณิตศาสตร์: ปัจจุบันระบบวิเคราะห์เชิงสัญลักษณ์ (Symbolic Execution Engine) ที่เป็นจุดแข็งหลักของ Symflower นั้น รองรับแบบสมบูรณ์เฉพาะภาษา Java, Spring, Spring Boot และ Go เท่านั้น
ปัญหาเส้นทางระเบิด (Path Explosion): หากโค้ดมีความซับซ้อนอย่างมหาศาล มีลูปที่ซ้อนกันจำนวนมาก
เส้นทางการทำงานที่เป็นไปได้จะเพิ่มขึ้นเป็นทวีคูณ ส่งผลให้ตัวคำนวณคณิตศาสตร์อาจใช้เวลาประมวลผลนานขึ้น

หรือทำให้ระบบใช้ทรัพยากรเครื่องสูง (แม้ว่า Symflower จะใช้ระบบ Pruning

และอัลกอริทึมค้นหาแบบพิเศษมาช่วยลดปัญหาก็คตาม)

ข้อจำกัดเกี่ยวกับไลบรารีภายนอก: ระบบวิเคราะห์เชิงคณิตศาสตร์ยังไม่สามารถจำลองความเข้าใจในไลบรารีภายนอก (External Dependencies) หรือไลบรารีมาตรฐานบางตัวของภาษาได้อย่างครบถ้วน 100%

ซึ่งอาจทำให้ในบางกรณีระบบสร้างชุดทดสอบได้ไม่สมบูรณ์

ฟังก์ชันขั้นสูงต้องใช้ใบอนุญาตแบบชำระเงิน: ถึงแม้ว่าจะมีเวอร์ชันให้ทดลองใช้งานได้ฟรี

แต่ฟังก์ชันที่สำคัญสำหรับการใช้งานในระดับองค์กรจำเป็นต้องซื้อใบอนุญาต (License)

ไม่สามารถเขียนลอจิกหลักของโปรแกรมได้: Symflower มุ่งเน้นเฉพาะการวิเคราะห์โค้ด การสร้างชุดทดสอบ

และการซ่อมแซมโค้ดเท่านั้น

มันจึงไม่สามารถช่วยเขียนโค้ดระบบหลัก (Business Logic) ขึ้นมาใหม่ตั้งแต่ต้นเหมือนเครื่องมือกลุ่ม GitHub Copilot ได้

ตัวอย่างการใช้งาน

เพื่อแสดงให้เห็นภาพว่า Symflower

เปลี่ยนโค้ดโปรแกรมมีงธรรมดาให้กลายเป็นระบบสมการคณิตศาสตร์เพื่อหาค่าทดสอบได้อย่างไร

ลองดูตัวอย่างโค้ดภาษา Java ที่มีโครงสร้างเงื่อนไขที่น่าสนใจดังต่อไปนี้ครับ

```
int twice(int v) {
```

```
    return 2 * v;
```

```
}
```

```
void testme(int x, int y) {
```

```
    int z = twice(y);
```

```
    if (z == x) {
```

```
        if (x > y + 10) {
```

```
            throw new RuntimeException("ERROR"); // จุดบกพร่องที่เราต้องการให้เทสต์เคสวิ่งมาถึง
```

```
        }
```

```
    }
```

```
}
```

ขั้นตอนการแปลงลอจิกเป็นสมการคณิตศาสตร์ของ Symflower

เมื่อคุณสั่งให้ Symflower วิเคราะห์ฟังก์ชัน testme เครื่องมือนี้จะจำลองการทำงานของโค้ดแบบประมวลผลเชิงสัญลักษณ์ (Symbolic Execution) โดยมีกลไกการเปลี่ยนโค้ดเป็นคณิตศาสตร์ที่ละขั้นตอนดังนี้

1. การกำหนดค่าเริ่มต้นด้วยสัญลักษณ์ (Symbolic Initialization)

แทนที่จะส่งตัวเลขจริง (เช่น x=5,y=10) เข้าไปรันตรงๆ Symflower จะกำหนดค่าตัวแปรนำเข้าให้เป็น "ค่าสัญลักษณ์สมมติ"

โดยจะแปลงตัวแปรให้อยู่ในรูปแบบ Static Single Assignment (SSA) ซึ่งจะทำให้การใส่เวอร์ชันกำกับตัวแปรย่อย (เช่น x_0, y_0) เพื่อให้ค่าของพวกมันคงที่และไม่ถูกเขียนทับในทางคณิตศาสตร์

ตัวแปร x จะถูกแทนด้วย x_0

ตัวแปร y จะถูกแทนด้วย y_0

2. การติดตามเส้นทางโค้ดและสะสมเงื่อนไข (Path Tracing & Constraint Assembly)

เมื่อจำลองการรันโปรแกรมไปที่ละบรรทัด Symflower จะบันทึกทุกลอจิกการทำงานและคำสั่งเงื่อนไขสะสมเข้าไปใน

"เงื่อนไขเส้นทาง" (Path Constraints หรือ π):

บรรทัด `int z = twice(y);` เนื่องจากฟังก์ชัน `twice` ทำการส่งค่า $2 \cdot v$ กลับคืนมา Symflower

จะกำหนดสมการของตัวแปร z_0 ขึ้นมาทันที

$$z_0 = 2 \cdot y_0$$

ทางแยกเงื่อนไขแรก `if (z == x):` ลอจิกจะถูกแตกสาขาออกเป็นสองทาง (Path Splitting) เพื่อหาทางที่เป็นจริงและเท็จ กรณีที่ไม่เป็นจริง (False):

$$z_0 \neq x_0$$

(โปรแกรมจบการทำงานปกติ)

กรณีที่เป็นจริง (True): เงื่อนไข $z_0 = x_0$

จะถูกบันทึกสะสมไว้ในระบบสมการเส้นทาง

ส่งผลให้สมการปัจจุบันคือ:

$$\pi = (2 \cdot y_0 = x_0)$$

ทางแยกเงื่อนไขที่สอง `if (x > y + 10):` บนเส้นทางที่เป็นจริงของเงื่อนไขแรก โปรแกรมจะเดินทางต่อมาถึงเงื่อนไขที่สอง

หากเราต้องการจะวิ่งไปถึงจุดผิดพลาดเพื่อสร้างเทสต์เคสจำลอง บรรทัดนี้จะต้องประเมินค่าเป็นจริงเท่านั้น

สมการสะสมจึงถูกอัปเดตเป็น

:

$$\pi = (2 \cdot y_0 = x_0) \wedge (x_0 > y_0 + 10)$$

3. การแก้โจทย์ด้วยคณิตศาสตร์ชั้นสูง (SMT Solving)

เมื่อเก็บรวบรวมเงื่อนไขเส้นทางจนถึงเป้าหมายแล้ว Symflower จะจัดรูปแบบสมการเหล่านั้นให้อยู่ในภาษามาตรฐานที่เรียกว่า

SMT-LIB และส่งต่อให้โปรแกรมแก้สมการคณิตศาสตร์ระดับลึก (เช่น Microsoft Z3 หรือ SMT Solver อื่นๆ)

เพื่อคำนวณหาคำตอบ:

$$\begin{cases} x_0 = 2 \cdot y_0 \\ x_0 > y_0 + 10 \end{cases}$$

4. สร้างค่าทดสอบรูปธรรม (Concrete Value Generation)

ตัว Solver จะทำการคำนวณเพื่อหาตัวเลขอินพุตจริงที่จะสามารถพิสูจน์ระบบสมการนี้ให้เป็นจริงได้สำเร็จ

หาก Solver สุ่มเลือกค่าเริ่มต้นให้ $y_0 = 11$

จากสมการแรก $x_0 = 2 \cdot 11 \Rightarrow x_0 = 22$

เมื่อทดลองนำไปแทนค่าในสมการที่สอง $22 > 11 + 10 \Rightarrow 22 > 21$ (ซึ่งเป็นจริง!)

ระบบสมการนี้จึงประเมินค่าเป็น Satisfiable (SAT) หรือหาคำตอบได้สำเร็จ Symflower จึงนำตัวเลขจริง $x = 22$ และ

$y = 11$ ที่ได้จากการถอดสมการนี้ ไปเขียนโครงร่างยูนิทเทสต์ (JUnit/Go-test) ให้คุณโดยอัตโนมัติ

เพื่อยืนยันว่าอินพุตชุดนี้จะทำให้เกิด Exception ขึ้นจริงในโปรแกรม

NSGA-II

1. แนวคิดพื้นฐาน (Core Concept)

Genetic Algorithm (GA) แบบดั้งเดิม ตัดสินความดี-ไม่ดีของแต่ละ individual ด้วย fitness function เพียงค่าเดียว ทำให้เปรียบเทียบกันได้ง่ายตรงไปตรงมา แต่ในปัญหาจริงจำนวนมาก มักมี "เป้าหมายหลายด้านที่ขัดแย้งกัน" (conflicting objectives) พร้อมกัน เช่น ในงาน Automated Test Case Generation อาจต้องการทั้ง code coverage สูงสุด และจำนวน test case น้อยที่สุดไปพร้อมกัน ซึ่งสองเป้าหมายนี้มักไม่สามารถดีที่สุดพร้อมกันได้ทั้งคู่

NSGA-II (Non-dominated Sorting Genetic Algorithm II) คือ Genetic Algorithm

ที่ถูกออกแบบมาเพื่อแก้ปัญหานี้โดยเฉพาะ กล่าวคือเป็น Multi-Objective Evolutionary Algorithm (MOEA)

ที่ใช้กลไกพื้นฐานของ GA (การคัดเลือก การผสมพันธุ์ การกลายพันธุ์) เหมือนเดิมทุกประการ แต่เปลี่ยน "วิธีตัดสินว่าใครดีกว่าใคร"

จากการเทียบค่าคะแนนเดียว มาเป็นการเทียบกันในทุกมิติพร้อมกัน

2. คำศัพท์เฉพาะ (Terminology)

Pareto Dominance: ความสัมพันธ์ที่บอกว่า individual A "ครอบงำ" (dominate) individual B ได้ก็ต่อเมื่อ A ดีกว่าหรือเท่ากับ B ในทุกมิติ (objective) และดีกว่าจริงอย่างน้อย 1 มิติ ถ้า A ดีกว่าในมิติหนึ่ง แต่ B ดีกว่าในอีกมิติหนึ่ง จะถือว่าไม่มีใครครอบงำใครได้

Pareto Front: กลุ่มของ solution ทั้งหมดที่ไม่มีใครถูกครอบงำได้ (non-dominated) คือกลุ่ม "ตัวเลือกที่ดีที่สุดเท่าที่จะทำได้" ในภาพรวม โดยแต่ละ solution ในกลุ่มนี้อาจเก่งคนละด้านกัน ไม่มี solution เดียวที่ดีที่สุดตายตัว

3. หลักการทำงานหลัก (Key Concepts)

3.1 Non-dominated Sorting

เป็นเทคนิคที่ใช้จัดกลุ่ม (classify) population ทั้งหมดออกเป็นระดับชั้นของ Pareto Front โดยมีขั้นตอนดังนี้

(Initialization) กำหนด rank เริ่มต้นให้ทุก solution และสร้าง list วางสำหรับแต่ละ front

(Domination Check) แต่ละ solution ตรวจสอบว่ามีใครครอบงำมันบ้าง และมันครอบงำใครบ้าง

(Front Assignment) solution ที่ไม่ถูกใครครอบงำเลย จะถูกจัดเข้า Front ที่ 1 จากนั้นตัด Front 1 ออกไป

แล้วทำซ้ำกับกลุ่มที่เหลือเพื่อหา Front 2, 3, 4 ต่อไปเรื่อย ๆ

3.2 Crowding Distance

เป็นค่าที่ใช้รักษาความหลากหลาย (diversity) ของ solution ภายใน Pareto Front เดียวกัน เพื่อไม่ให้ solution กระจุกตัวอยู่บริเวณเดียวของพื้นที่คำตอบ มีขั้นตอนคำนวณดังนี้

Initialization: ตั้งค่า crowding distance ของทุก solution เป็นศูนย์

Sorting: สำหรับแต่ละ objective เรียงลำดับ solution ตามค่าของ objective นั้น

Boundary Assignment: solution ที่อยู่ริมสุดสองข้าง กำหนดค่า crowding distance เป็นอนันต์ (เพื่อให้ถูกเก็บไว้เสมอ)

Distance Calculation: สำหรับ solution อื่น คำนวณค่าจากผลต่างที่ normalize แล้วของ solution ข้างเคียงในแต่ละ objective

solution ที่มีค่าโดดเดียวต่ำ จะมีค่า crowding distance ต่ำ และมีโอกาสถูกตัดออกก่อนเมื่อ Front ใดมีจำนวน solution เกินโควตา ในทางกลับกัน solution ที่อยู่โดดเดียวจะมีค่าสูงและถูกเก็บไว้ก่อน เพื่อรักษาความหลากหลายของ solution set โดยรวม

3.3 Fast Non-dominated Sorting Algorithm

เป็นเทคนิคการคำนวณที่ทำให้ขั้นตอน Non-dominated Sorting (หัวข้อ 3.1) ทำงานได้เร็วขึ้นอย่างมีนัยสำคัญ

เมื่อเทียบกับวิธีดั้งเดิมที่ใช้ใน NSGA รุ่นแรก โดยมีหลักการดังนี้

Dominance Count: สำหรับแต่ละ solution นับจำนวน solution ที่ครอบงำมัน (dominance count) และเก็บรายชื่อ solution ที่มันครอบงำไว้ (dominated set)

First Front Assignment: solution ที่มี dominance count เท่ากับศูนย์ (ไม่มีใครครอบงำ) จะถูกจัดเข้า Front แรกทันที

Subsequent Fronts: สำหรับแต่ละ solution ใน Front ปัจจุบัน ลด dominance count ของ solution ที่มันครอบงำลง 1 หาก dominance count ของตัวใดกลายเป็นศูนย์ ให้จัดเข้า Front ถัดไป

ทำซ้ำขั้นตอนนี้จนกว่า solution ทุกตัวจะถูกจัดเข้า Front เรียบร้อย

เทคนิคนี้ลดความซับซ้อนในการคำนวณจาก $O(MN^3)$ ของวิธีดั้งเดิม ลงเหลือ $O(MN^2)$ โดยที่ M คือจำนวน objective และ N คือขนาดของ population ซึ่งเป็นเหตุผลหลักที่ทำให้ NSGA-II มีคำว่า "Fast" อยู่ในชื่อรุ่นต่อ ยอด

บทที่ 3

วิธีดำเนินงาน

3.1 ภาพรวมการดำเนินงาน

โครงการนี้ศึกษาการสร้างชุดทดสอบอัตโนมัติสำหรับโปรแกรมภาษา Java ด้วย 4 วิธี ได้แก่ NSGA-II ผ่าน EvoSuite, Symbolic Execution ผ่าน Symflower, ChatGPT และ GitHub Copilot โดยใช้ข้อบกพร่องจาก Defects4J จำนวน 854 กรณีครอบคลุม 17 โครงการ

การดำเนินงานแบ่งเป็นการเตรียมโปรแกรมและข้อมูลข้อบกพร่อง การสร้างชุดทดสอบ การตรวจสอบการคอมไพล์และการทำงาน การประเมินความสามารถในการตรวจพบข้อบกพร่อง และการวัดความครอบคลุมของโค้ด ผลการทดลองจัดเก็บพร้อมชุดทดสอบ configuration และหลักฐานที่เกี่ยวข้อง เพื่อให้สามารถตรวจสอบย้อนกลับถึงกรณีทดลองและชุดการรันได้

การรายงานใช้ตัวชี้วัดหลักร่วมกัน ได้แก่ ความพร้อมในการประเมินชุดทดสอบ การตรวจพบข้อบกพร่อง ความครอบคลุมของโค้ด และเวลา ส่วนข้อมูลเฉพาะวิธี เช่น search budget, จำนวน test declarations และ token usage รายงานแยกจากตัวชี้วัดร่วม

3.2 ชุดข้อมูลที่ใช้ในการทดลอง

Defects4J จัดเตรียมโปรแกรมสองรุ่นสำหรับแต่ละกรณี ได้แก่ รุ่นที่มีข้อบกพร่องหรือ buggy version และรุ่นที่แก้ไขข้อบกพร่องแล้วหรือ fixed version โดยใช้ชื่อโครงการและหมายเลขข้อบกพร่อง หรือ Project-Bug ID เป็นหน่วยการทดลอง

ตารางที่ 1 ชุดข้อมูลที่ใช้ในการทดลอง

โครงการ	จำนวนกรณี
Chart	26
Cli	39
Closure	174
Codec	18
Collections	28
Compress	47
Csv	16
Gson	18
JacksonCore	26
JacksonDataBind	110
JacksonXml	6

ตารางที่ 1 ชุดข้อมูลที่ใช้ในการทดลอง(ต่อ)

Jsoup	93
JXPath	22
Lang	61
Math	106
Mockito	38
Time	26
รวม	854

จำนวนกรณีทดลองนับจาก active bugs ที่ไม่ซ้ำกัน รายการ deprecated และรายการรันซ้ำไม่ถูกนับเป็นกรณีใหม่ ตัวอย่างเช่นข้อมูลรวมของ Symflower มี 857 แถว ประกอบด้วย active bugs 854 กรณี และ deprecated bugs ของ Lang อีก 3 แถว ซึ่งไม่รวมในการคำนวณผลหลัก

3.3 เครื่องมือและสภาพแวดล้อม

3.3.1 เครื่องมือหลัก

ตารางที่ 2 ตารางเครื่องมือหลัก

วิธี	เครื่องมือหรือบริการ	อัลกอริทึมหรือโมเดล
NSGA-II	EvoSuite 1.2	NSGA-II
Symbolic Execution	Symflower CLI	การวิเคราะห์และสร้างชุดทดสอบด้วย Symbolic Execution
ChatGPT	ระบบเรียกใช้งาน ChatGPT ผ่าน AI runner และ KGU IntelSphere API	gpt-5.6-terra
GitHub Copilot	GitHub Copilot CLI แบบ BYOK	DeepSeek-v4-pro

Defects4J ใช้สำหรับเตรียมโปรแกรม คอมไพล์ รันทดสอบ และประเมิน coverage ส่วน JUnit ใช้เป็น framework สำหรับชุดทดสอบ Java โดย dependency ที่ใช้จริงขึ้นอยู่กับโครงการเป้าหมาย

3.3.2 การกำหนดค่าของแต่ละวิธี

ตารางที่ 3 ตารางกำหนดค่าของแต่ละวิธี

วิธี	การกำหนดค่าหลัก
NSGA-II รอบที่ 1	อัลกอริทึม NSGAII, seed 11 และ search budget 150 วินาทีต่อคลาส
NSGA-II รอบที่ 2	อัลกอริทึม NSGAII, seed 11 และ search budget 60 วินาทีต่อคลาส
Symflower	โหมด plain Java, JUnit4 และเลือกขอบเขตจาก modified classes; มีการบันทึก memory limit 2,560 และ 4,096 MB ในชุดการทดลอง
ChatGPT	กระบวนการ P01-P04, source budget 12,000 ตัวอักษร และการซ่อมด้วย AI ไม่เกิน 1 ครั้ง
GitHub Copilot	ใช้ prompt templates และกติกากลยุทธ์เดียวกับ ChatGPT

3.4 ขั้นตอนการทดลองร่วม

กระบวนการหลักประกอบด้วยขั้นตอนต่อไปนี้

- 3.4.1 เตรียม buggy version และ fixed version ตาม Project-Bug ID
- 3.4.2 อ่าน metadata เช่น modified classes และ developer trigger tests
- 3.4.3 สร้างชุดทดสอบด้วยวิธีที่กำหนด
- 3.4.4 จัดเก็บ source ของชุดทดสอบและข้อมูลการรัน
- 3.4.5 ตรวจสอบการคอมไพล์และผลการทำงานบนโปรแกรมสองรุ่น
- 3.4.6 ประเมินการตรวจพบข้อบกพร่องตามทิศทางที่ใช้
- 3.4.7 วัด coverage ในกรณีที่มีผลประเมิน
- 3.4.8 รวบรวมสถานะ ผลลัพธ์ เวลา และหลักฐานประกอบ
- 3.4.9 ผลจาก developer tests แยกออกจากผลของชุดทดสอบที่สร้างขึ้น โดยเฉพาะ developer trigger tests ซึ่งมีการล้มเหลวบน buggy version อยู่แล้ว การล้มเหลวดังกล่าวจึงไม่ถือเป็นผลงานการตรวจพบของเครื่องมือสร้างชุดทดสอบ

3.5 การสร้างชุดทดสอบด้วย NSGA-II

การทดลองใช้ EvoSuite โดยกำหนดอัลกอริทึมเป็น NSGA-II เพื่อสร้างชุดทดสอบสำหรับ modified classes ของแต่ละกรณี ตัว generator ใช้ classpath จาก buggy version และจัดชุดทดสอบเป็น archive สำหรับนำไปประเมินด้วย Defects4J

การใช้ NSGA-II ในการทดลองนี้เป็นการค้นหาและสร้างชุดทดสอบภายใน EvoSuite ไม่ใช่การคัดเลือกจาก test pool ที่รวมผลของเครื่องมือทั้ง 4 วิธี

ทดลองสองรอบโดยใช้ search budget 150 และ 60 วินาทีต่อคลาสตามลำดับ และใช้ seed 11 ชุดทดสอบที่สร้างขึ้นถูกนำไปประเมินบนโปรแกรมสองรุ่น ผลตรวจพบที่บันทึกในข้อมูลนี้ใช้ทิศทาง ผ่านบน buggy version และล้มเหลวบน fixed version เพื่อแสดงว่าชุดทดสอบสามารถแยกพฤติกรรมก่อนและหลังการแก้ไขได้

รอบที่ 2 มีข้อมูลจากสองเครื่องรวม 940 รายการ ครอบคลุม 854 กรณี โดยมี 86 กรณีที่ถูกบันทึกสองครั้ง การรวมผลเลือกการรันล่าสุดที่มีสถานะ ok หากไม่มีการรันสำเร็จจึงใช้รายการล่าสุดของกรณีนั้น ผลรวมจึงเป็นหนึ่งรายการต่อ Project-Bug ID

3.6 การสร้างชุดทดสอบด้วย Symbolic Execution ผ่าน Symflower

การทดลองใช้ Symflower วิเคราะห์โค้ด Java โดยเลือก package ที่เกี่ยวข้องกับ modified classes และมีการจำกัดเป็น class scope ในบางกรณี ใช้โหมด plain Java และกำหนด test framework เป็น JUnit4 เพื่อให้สอดคล้องกับ build ของโครงการที่ทดลอง

กระบวนการบันทึกจำนวนไฟล์ชุดทดสอบและจำนวน @Test declarations จากนั้นนำไฟล์ที่สร้างขึ้นไปตรวจสอบบน buggy version และ fixed version

การประเมิน fault detection พิจารณา generated tests ที่ล้มเหลวบน buggy version และผ่านบน fixed version โดยตัด developer trigger tests ออก การอ่าน metadata รองรับตัวค้นรายชื่อที่แตกต่างกันระหว่างโครงการ

ค่า coverage ในข้อมูลหลักของ Symflower มาจากการรันที่รวม developer tests จึงเป็นผลของชุดทดสอบรวม ไม่ใช่ความครอบคลุมของ generated tests เพียงอย่างเดียว

มีการทดลองเพิ่มเติมโดยกำหนด timeout 60 วินาทีต่อเมธอดในตัวอย่าง Lang จำนวน 8 กรณี และวิเคราะห์เฉพาะคลาสใน Lang-1 ผลเพิ่มเติมเหล่านี้แยกจากผลหลักของ 854 กรณี

3.7 การสร้างชุดทดสอบด้วย ChatGPT และ GitHub Copilot

ทั้งสองวิธีใช้ prompt templates เหมือนกันทั้ง 4 ขั้นตอน และใช้กติกาการเลือกบริบทจาก buggy version ชุดเดียวกัน

ตารางที่ 4 ตารางรายละเอียดแต่ละ Prompt

ขั้นตอน	การดำเนินการ
P01 วิเคราะห์บริบท	ให้ข้อมูลข้อบกพร่องและ API signatures เพื่อสร้างแผนทดสอบแบบย่อ โดยขอคำตอบไม่เกิน 10 บรรทัด
P02 สร้างชุดทดสอบ	ให้ source/context ที่คัดเลือกแบบ deterministic ไม่เกิน 12,000 ตัวอักษร พร้อมแผนจาก P01 และขอไม่เกิน 12 test methods
P03 ซ่อมชุดทดสอบ	เมื่อ validation ไม่ผ่าน ส่งชุดทดสอบและข้อความผิดพลาดแบบย่อให้ AI แก้ไขได้ 1 ครั้ง
P04 เพิ่มความครอบคลุม	ใช้ coverage ที่วัดได้จริงและ source snippets เพื่อขอเพิ่มไม่เกิน 4 tests หรือให้ตอบ NO_CHANGE

จำนวนเมธอดที่ระบุใน prompt เป็นคำขอที่ส่งให้โมเดล ไม่ใช่ข้อรับประกันว่าคำตอบทุกกรณีมีจำนวนตรงตามนั้น Fixed source ไม่ถูกส่งให้ AI เพื่อสร้างชุดทดสอบ แต่ใช้ตรวจสอบผลที่สร้างขึ้น ชุดทดสอบที่ผ่าน fixed validation จึงถูกนำไปประเมินกับ buggy version เพื่อวัด fault detection

Protocol มีการปรับรูปแบบ P03 จาก unified diff เป็น Java class ฉบับเต็ม และปรับ P04 ให้ปฏิเสธส่วนขยายที่ไม่ผ่านการตรวจสอบ โดยคงชุดเดิมที่ผ่าน validation ไว้ ผลย้อนหลังอ้างอิงขั้นตอนที่บันทึกใน artifacts ของแต่ละกรณี

ชุดผลหลักของ ChatGPT คือ run-final-opt และของ GitHub Copilot คือ run-copilot-final-v2 ผลจากชุดการทดลองก่อนหน้านี้แยกออกจากข้อมูลสรุปหลัก

3.8 เกณฑ์การประเมินผล

3.8.1 ความพร้อมในการประเมินชุดทดสอบ

แยกการมีไฟล์ทดสอบ การคอมไพล์ การรัน และการผ่าน fixed validation ออกจากกัน เนื่องจากการผ่านขั้นตอนหนึ่งไม่ยืนยันว่าจะผ่านขั้นตอนถัดไป

ตารางที่ 5 ตารางรายละเอียดความหมายของสถานะที่ใช้รายงาน

วิธี	ความหมายของสถานะที่ใช้รายงาน
NSGA-II	ok หมายถึงมีผลประเมินบนโปรแกรมสองรุ่น ไม่ได้หมายถึงผ่าน fixed version ทุกกรณี
Symflower	การมีไฟล์และไม่ระบุ compile fail เป็นสถานะความคืบหน้าตาม CSV ไม่ใช่การยืนยัน fixed validation
ChatGPT	DONE หมายถึงผ่าน fixed validation และมี final evaluation หรือ test-only recovery
GitHub Copilot	ใช้นิยาม DONE เช่นเดียวกับ ChatGPT

3.8.2 การตรวจพบข้อบกพร่อง

ตารางที่ 6 ตารางการตรวจพบข้อบกพร่อง

วิธี	ทิศทางผลที่ใช้
NSGA-II	ชุดทดสอบผ่านบน buggy และล้มเหลวบน fixed
Symflower ในผลหลัก	Generated test ล้มเหลวบน buggy และผ่านบน fixed โดยตัด developer trigger tests
ChatGPT	ชุดทดสอบผ่านบน fixed และมี test ล้มเหลวบน buggy
GitHub Copilot	ชุดทดสอบผ่านบน fixed และมี test ล้มเหลวบน buggy

3.8.3 ความครอบคลุมของโค้ด

ใช้ชื่อ Line coverage และ Condition coverage ตามค่าที่เครื่องมือรายงาน

$$Line\ coverage\ (\%) = \frac{LinesCovered}{LinesTotal} \times 100$$

$$Condition\ coverage\ (\%) = \frac{ConditionsCovered}{ConditionsTotal} \times 100$$

โดย	LinesCovered	คือจำนวนบรรทัดที่ถูกเรียกใช้งาน
	LinesTotal	คือจำนวนบรรทัดทั้งหมดที่เครื่องมือใช้ประเมิน
	ConditionsCovered	คือจำนวนเงื่อนไขที่ถูกครอบคลุม
	ConditionsTotal	คือจำนวนเงื่อนไขทั้งหมดที่เครื่องมือใช้ประเมิน

ค่าเฉลี่ยคำนวณจากเปอร์เซ็นต์รายการที่มีข้อมูลจริง ไม่ใช่การรวมจำนวน covered และ total ของทุกกรณีแล้วหารครั้งเดียว ค่าที่ไม่มีข้อมูลไม่แทนด้วยศูนย์

3.8.4 จำนวนการทดสอบ

จำนวน test declarations ที่นับจาก source แยกจากจำนวน executions ที่ test runner รันจริง การมี declarations ไม่ยืนยันว่าการทดสอบทั้งหมดคอมไพล์ได้หรือถูกเรียกใช้งาน

3.8.5 เวลาและทรัพยากร

เวลาแต่ละประเภทระบุขั้นตอนที่ครอบคลุมและจำนวนกรณีที่มีข้อมูล เวลาสร้างชุดทดสอบ เวลาเรียกบริการ AI และเวลาประเมินผลไม่ถือเป็นเวลาเดียวกัน

Token ใช้อธิบายทรัพยากรของบริการ AI ส่วน search budget ใช้อธิบายการกำหนดเวลาค้นหาของ NSGA-II ทั้งสองประเภทเป็นข้อมูลเฉพาะวิธี

3.9 การจัดการข้อผิดพลาดและการประเมินซ้ำ

แยกข้อผิดพลาดของสภาพแวดล้อมและระบบออกจากผลที่เครื่องมือสร้างไม่สำเร็จ เช่น การเชื่อมต่อขัดข้อง การประเมินไม่ครบ และ instrumentation ล้มเหลว เป็นคนละประเภทกับชุดทดสอบที่คอมไพล์ไม่ผ่านหรือมี assertions ไม่ถูกต้อง สำหรับ AI ปัญหาระบบสามารถ retry หรือประเมินซ้ำเพื่อกู้หลักฐานได้ ส่วนผลของโมเดลที่ไม่สำเร็จไม่ถูกเรียกใหม่เพียงเพื่อให้ได้ผลที่ดีขึ้น

ผลการประเมินซ้ำเชื่อมโยงกับชุดทดสอบเดิม โดยใช้ source hash และ archive ช่วยตรวจสอบตัวตนของชุดทดสอบ พร้อมเก็บหลักฐานเดิมไว้เมื่อพบความไม่สอดคล้อง

3.10 การจัดเก็บผลการทดลอง

ตารางที่ 7 ตารางรายละเอียดตำแหน่งการจัดเก็บผลการทดลอง

ตำแหน่ง	ข้อมูล
Algorithm1_NSGAII/	โค้ด generator และ runner ชุดทดสอบ หลักฐาน และผลสองรอบ
Algorithm2_SymbolicExecution/	โค้ด configuration ชุดทดสอบ ผลรายการณี และ diagnostics
AI1_ChatGPT/	Prompt คำตอบ ชุดทดสอบ metadata และผลของ ChatGPT
AI2_GitHubCopilot/	Prompt คำตอบ ชุดทดสอบ metadata และผลของ GitHub Copilot
Experiment/automation/	สคริปต์ดำเนินการ ประเมิน และสร้างข้อมูลสรุป
Experiment/evaluations/	หลักฐานการประเมินชุดทดสอบ
Report/	ตารางข้อมูลและเอกสารสรุปผล

ข้อมูลแต่ละรายการอ้างอิง Project-Bug ID และชุดการรันเดียวกัน เพื่อป้องกันการนำชุดทดสอบกับผลประเมินจากคนละการรันมารวมกัน

บทที่ 4

ผลการดำเนินงาน

4.1 ภาพรวมผลการทดลอง

ทั้ง 4 วิธีใช้ขอบเขตการทดลอง 854 กรณีเท่ากัน แต่มีผลการสร้างและประเมินชุดทดสอบต่างกัน โดยสถานะที่รายงานของแต่ละวิธีมีความหมายเฉพาะตามกระบวนการทดลอง

ตารางที่ 8 ตารางภาพรวมผลการทดลอง

วิธี	กรณีทั้งหมด	ผลตามสถานะของแต่ละวิธี	สัดส่วนต่อกรณีทั้งหมด
NSGA-II รอบที่ 1	854	ok 789	92.39%
NSGA-II รอบที่ 2	854	ok 669	78.34%
Symflower	854	มีไฟล์และไม่ระบุ compile fail 49	5.74%
ChatGPT	854	DONE 561	65.69%
GitHub Copilot	854	DONE 229	26.81%

สถานะในตารางมีความหมายตามกระบวนการของแต่ละวิธี จึงไม่ใช่จำนวนดังกล่าวแทนความถูกต้องบน fixed version ประเภทเดียวกันทั้งหมด

4.2 ผลการทดลองด้วย NSGA-II

4.2.1 ผลตามสถานะ

ตารางที่ 9 ตารางผลการทดลองด้วย NSGA-II ข้อมูลตามสถานะ (1)

ผลลัพธ์	รอบที่ 1	รอบที่ 2
ประเมินได้ (ok)	789	669
สร้างชุดทดสอบไม่สำเร็จ	34	165
ไม่สามารถประเมินได้	31	17
คอมไพล์ buggy version ไม่ผ่าน	0	3
รวม	854	854

กรณีที่มีสถานะ ok แบ่งตามผลบนโปรแกรมสองรุ่นได้ดังนี้

ตารางที่ 10 ตารางผลการทดลองด้วย NSGA-II ข้อมูลตามสถานะ (2)

ผลทดสอบ	รอบที่ 1	รอบที่ 2
ผ่านทั้ง buggy และ fixed	536	459
ล้มเหลวทั้ง buggy และ fixed	142	115
ผ่าน buggy และล้มเหลวนบน fixed	111	95
รวม	789	669

จำนวนกรณีที่รายงาน fault_detected เท่ากับ 111 และ 95 กรณีตามลำดับ โดยทั้งหมดมีทิศทางผ่านบน buggy และล้มเหลวนบน fixed จึงแสดงความแตกต่างของพฤติกรรมตามเกณฑ์ที่ใช้ในชุดข้อมูลนี้

4.2.2 ผลแยกตามโครงการ

ตารางที่ 11 ตารางผลการทดลองด้วย NSGA-II ข้อมูลตามโครงการ

โครงการ	กรณีทั้งหมด	ประเมินได้ รอบ 1	ตรวจพบ รอบ 1	ประเมินได้ รอบ 2	ตรวจพบ รอบ 2
Chart	26	23	3	23	1
Cli	39	29	1	28	0
Closure	174	168	7	135	4
Codec	18	17	4	17	4
Collections	28	28	0	26	1
Compress	47	41	6	45	7
Csv	16	16	7	15	5
Gson	18	17	3	17	1
JacksonCore	26	25	8	24	5
JacksonDataBind	110	104	11	66	6
JacksonXml	6	6	1	5	1
Jsoup	93	85	18	77	23
JXPath	22	17	2	18	4
Lang	61	55	7	45	8
Math	106	99	28	71	19
Mockito	38	33	2	32	2
Time	26	26	3	25	4
รวม	854	789	111	669	95

รอบที่ 1 มีจำนวนที่ตรวจพบมากที่สุดใน Math ส่วนรอบที่ 2 มีจำนวนมากที่สุดใน Jsoup ผลไม่ได้เปลี่ยนไปในทิศทางเดียวกันทุกโครงการเมื่อเปลี่ยน search budget

4.2.3 ผลการรันซ้ำ

รอบที่ 2 มี 86 กรณีที่ถูกบันทึกสองครั้ง พบว่า 43 กรณีมีผลของ `buggy_result`, `fixed_result`, `fault_detected` หรือ `status` ต่างกัน และ 48 กรณีมี `coverage` ต่างกัน

ข้อมูลสรุปใช้กติกาลือการรันล่าสุดที่สำเร็จ จึงเป็นผลหลังคัดเลือก ไม่ใช่ค่าเฉลี่ยจากการรันซ้ำทั้งหมด

4.3 ผลการทดลองด้วย Symflower

4.3.1 ผลตามขั้นตอน

ตารางที่ 12 ตารางผลการทดลองด้วย Symflower ข้อมูลตามขั้นตอน

ผลลัพธ์	จำนวนกรณี	สัดส่วนต่อกรณีทั้งหมด
Checkout ไม่สำเร็จ	26	3.04%
ไม่มีไฟล์ชุดทดสอบที่สร้างขึ้น	672	78.69%
มีไฟล์ชุดทดสอบ แต่ระบุ compile fail	107	12.53%
มีไฟล์ชุดทดสอบ และไม่ระบุ compile fail	49	5.74%
รวม	854	100.00%

มี 698 กรณีที่ไม่มีไฟล์ชุดทดสอบ โดยรวม checkout fail 26 กรณี จึงไม่ถือว่าทั้งหมดเป็นความล้มเหลวของการวิเคราะห์ด้วย Symflower โดยตรง

จาก 156 กรณีที่มีไฟล์ชุดทดสอบ มี 107 กรณีระบุ compile fail คิดเป็น 68.59% ของกลุ่มดังกล่าว ปัญหาที่รายงานเกี่ยวข้องกับ API, dependencies และความเข้ากันได้ของ test framework

อีก 49 กรณีเป็นผลตามสถานะ CSV ไม่ใช่จำนวนที่ยืนยันว่าผ่าน fixed validation ทั้งหมด เนื่องจากผลบางรายการรวม developer tests

4.3.2 ผลแยกตามโครงการ

ตารางที่ 13 ตารางผลการทดลองด้วย Symflower ข้อมูลตามโครงการ

โครงการ	กรณีทั้งหมด	กรณีที่มีไฟล์ทดสอบ	@Test declarations	กรณีที่ตรวจพบ
Chart	26	0	0	0
Cli	39	5	125	0
Closure	174	0	0	0
Codec	18	5	64	0
Collections	28	4	9	0
Compress	47	33	2,858	0

ตารางที่ 13 ตารางผลการทดลองด้วย Symflower ข้อมูลตามโครงการ (ต่อ)

Csv	16	6	18	0
Gson	18	0	0	0
JacksonCore	26	14	3,896	0
JacksonDatabind	110	8	42	0
JacksonXml	6	0	0	0
Jsoup	93	33	1,788	0
JXPath	22	0	0	0
Lang	61	22	1,510	0
Math	106	19	150	0
Mockito	38	0	0	0
Time	26	7	217	0
รวม	854	156	10,677	0

Symflower สร้างไฟล์ชุดทดสอบได้ใน 11 จาก 17 โครงการ รวม 156 กรณี คิดเป็น 18.27% ของกรณีทดลองทั้งหมด จำนวน @Test declarations ใน JacksonCore, Compress, Jsoup และ Lang รวม 10,052 รายการ หรือ 94.15% ของทั้งหมด ทั้งนี้ declarations เป็นการนับจาก source code ไม่ใช่จำนวน tests ที่รันสำเร็จจริง

4.3.3 การทดลองเพิ่มเติมและการตรวจสอบผล

การทดลอง timeout 60 วินาทีต่อเมธอดใน Lang 8 กรณีไม่พบการตรวจพบเพิ่มเติมตามผลที่รายงาน การทดลองนี้มีขอบเขตเฉพาะกลุ่มตัวอย่าง ไม่ใช่รอบที่สองครบ 854 กรณี

ผลเดิมของ Lang-19, Lang-20, Lang-22 และ Lang-36 เคยถูกนับว่าตรวจพบ แต่การตรวจสอบพบว่า developer trigger tests ถูกตัดออกไม่ครบ เพราะ parser รองรับตัวค้น , แต่บางรายการใช้ ; หลังแก้ไข ผล detection ของทั้งสี่กรณีเป็นศูนย์

4.4 ผลการทดลองด้วย ChatGPT

4.4.1 ผลตามสถานะ

ตารางที่ 14 ตารางผลการทดลองด้วย ChatGPT ข้อมูลตามสถานะ

ผลลัพธ์	จำนวนกรณี	สัดส่วนต่อกรณีทั้งหมด
ประเมินสำเร็จ (DONE)	561	65.69%
ไม่ผ่านหลังซ่อม	272	31.85%
ไม่ผ่านหลัง P04	8	0.94%
คำตอบไม่สมบูรณ์	13	1.52%
รวม	854	100.00%

กรณีที่ประเมินสำเร็จตรวจพบข้อบกพร่อง 403 กรณี ส่วนอีก 158 กรณีไม่มีผลตรวจพบตามเกณฑ์ที่ใช้ สถานะไม่ผ่านหลัง P04 เป็นผลย้อนหลังในชุดข้อมูล ไม่ได้หมายความว่าทุกกรณีใช้ protocol revision เดียวกัน

4.4.2 ผลแยกตามโครงการ

ตารางที่ 15 ตารางผลการทดลองด้วย ChatGPT ข้อมูลตามโครงการ

โครงการ	กรณีทั้งหมด	ประเมินสำเร็จ	กรณีที่ตรวจพบ fault
Chart	26	15	10
Cli	39	25	17
Closure	174	47	28
Codec	18	10	7
Collections	28	26	15
Compress	47	40	32
Csv	16	13	12
Gson	18	13	11
JacksonCore	26	19	12
JacksonDataBind	110	96	69
JacksonXml	6	2	2
Jsoup	93	62	44
JXPath	22	9	7
Lang	61	54	45
Math	106	77	53
Mockito	38	33	25
Time	26	20	14
รวม	854	561	403

ChatGPT มีจำนวนที่ตรวจพบมากที่สุดใน JacksonDataBind รองลงมาคือ Math และ Lang จำนวนดังกล่าวเป็นจำนวนข้อบกพร่องที่ตรวจพบ ไม่ใช่จำนวน test methods ที่ล้มเหลว

4.5 ผลการทดลองด้วย GitHub Copilot

4.5.1 ผลตามสถานะ

ตารางที่ 16 ตารางผลการทดลองด้วย GitHub Copilot ข้อมูลตามสถานะ

ผลลัพธ์	จำนวนกรณี	สัดส่วนต่อกรณีทั้งหมด
ประเมินสำเร็จ (DONE)	229	26.81%
ไม่ผ่านหลังซ่อม	599	70.14%
คำตอบไม่สมบูรณ์	26	3.04%
รวม	854	100.00%

กรณีที่ประเมินสำเร็จตรวจพบข้อบกพร่อง 166 กรณี ส่วนอีก 63 กรณีไม่มีผลตรวจพบตามเกณฑ์ที่ใช้

4.5.2 ผลแยกตามโครงการ

ตารางที่ 17 ตารางผลการทดลองด้วย GitHub Copilot ข้อมูลตามโครงการ

โครงการ	กรณีทั้งหมด	ประเมินสำเร็จ	กรณีที่ตรวจพบ
Chart	26	10	7
Cli	39	12	8
Closure	174	9	6
Codec	18	9	6
Collections	28	10	8
Compress	47	14	13
Csv	16	10	10
Gson	18	6	5
JacksonCore	26	14	11
JacksonDatabind	110	21	13
JacksonXml	6	2	0
Jsoup	93	25	18
JXPath	22	3	2
Lang	61	22	17
Math	106	37	22
Mockito	38	12	11
Time	26	13	9
รวม	854	229	166

GitHub Copilot มีจำนวนที่ตรวจพบมากที่สุดใน Math รองลงมาคือ Jsoup และ Lang

4.6 ผลเปรียบเทียบทั้ง 4 วิธี

4.6.1 การตรวจพบข้อบกพร่อง

ตารางที่ 18 ตารางผลเปรียบเทียบทั้ง 4 วิธี ข้อมูลตามการตรวจพบข้อบกพร่อง

วิธี	กรณีที่ตรวจพบ	ฐานผลประโยชน์	อัตราในกลุ่มที่มีผล	อัตราต่อ 854 กรณี
NSGA-II รอบที่ 1	111	789	14.07%	13.00%
NSGA-II รอบที่ 2	95	669	14.20%	11.12%
Symflower	0	—	—	0.00%
ChatGPT	403	561	71.84%	47.19%
GitHub Copilot	166	229	72.49%	19.44%

Symflower ไม่ใช้อีก 49 กรณีเป็นฐาน fixed validation เนื่องจากสถานะตาม CSV ไม่ยืนยันการผ่าน generated-test validation ทั้งหมด

อัตราต่อ 854 กรณีแสดงจำนวนที่มีหลักฐานตรวจพบเมื่อเทียบกับขอบเขตทั้งหมด ไม่ได้ตัดสินว่ากรณีที่ไม่มีผลประโยชน์เป็นชุดทดสอบที่ตรวจไม่พบ

ในสองวิธี AI ที่ใช้เกณฑ์ผ่านบน fixed และล้มเหลวบน buggy เหมือนกัน ChatGPT ตรวจพบ fault ได้จำนวนมากกว่าส่วน GitHub Copilot มีอัตราตรวจพบเฉพาะกลุ่ม DONE สูงกว่าเล็กน้อย อย่างไรก็ตาม กลุ่ม DONE มีจำนวนและองค์ประกอบต่างกัน จึงพิจารณาผลบนกรณีรวมในหัวข้อ 4.7.2 เพิ่มเติม

ผล NSGA-II ใช้ทิศทางผ่าน buggy และล้มเหลวบน fixed ขณะที่ AI ใช้ทิศทางตรงกันข้าม ตารางนี้จึงแสดงผลตามเกณฑ์ของแต่ละวิธี ไม่ใช่การจัดอันดับภายใต้นิยาม fixed-valid fault detection เดียวกันทั้งหมด

4.6.2 ความครอบคลุมของโค้ด

ตารางที่ 19 ตารางผลเปรียบเทียบทั้ง 4 วิธี ข้อมูลตามความครอบคลุมของโค้ด

วิธี	กรณีที่มี Line coverage	Line coverage เฉลี่ย	กรณีที่มี Condition coverage	Condition coverage เฉลี่ย
NSGA-II รอบที่ 1	789	54.27%	786	43.97%
NSGA-II รอบที่ 2	669	49.61%	666	39.19%
Symflower	—	—	—	—
ChatGPT	559	51.00%	556	42.60%
GitHub Copilot	227	48.29%	226	41.85%

ผล NSGA-II และ AI ในตารางวัดบน fixed version แต่ใช้กลุ่มกรณีที่มีข้อมูลของแต่ละวิธี ค่าเฉลี่ยไม่ได้คำนวณบน Project-Bug ID ชุดเดียวกันทั้งหมด

Coverage ของ Symflower ในข้อมูลหลักรวม developer tests และมีการวัดบน buggy version จึงไม่รวมในค่าเฉลี่ยช่องเดียวกับ generated-test coverage บน fixed version เครื่องหมาย “—” ไม่ได้หมายถึงศูนย์

4.6.3 เวลา

ตารางที่ 20 ตารางผลเปรียบเทียบทั้ง 4 วิธี ข้อมูลตามเวลา

วิธี	ขอบเขตเวลา	กรณีที่มีข้อมูล	เวลาเฉลี่ย
NSGA-II รอบที่ 1	เวลา generation ของกรณี ok	789	236.50 วินาที
NSGA-II รอบที่ 2	เวลา generation ของกรณี ok	669	119.01 วินาที
Symflower	เวลาเรียกคำสั่ง generation	828	97.69 วินาที
ChatGPT	เวลาเรียกบริการ AI รวม stages ที่เรียกจริง	854	83.40 วินาที
GitHub Copilot	เวลาเรียกบริการ AI รวม stages ที่เรียกจริง	854	387.50 วินาที

เวลา Symflower รวมกรณีที่ไม่มีไฟล์ทดสอบเมื่อมีเวลาบันทึก ส่วนเวลา NSGA-II ใช้เฉพาะกรณี ok และเวลา AI ไม่รวม compile/test/coverage เวลารอคิว และ retry ที่ไม่ได้อยู่ใน metadata ชุดนี้

มัธยฐานเวลา NSGA-II เท่ากับ 188.93 และ 95.68 วินาทีตามลำดับ ส่วน Symflower มีมัธยฐาน 15 วินาที ซึ่งต่ำกว่าค่าเฉลี่ย แสดงว่าบางกรณีใช้เวลามากและส่งผลต่อค่าเฉลี่ย ตารางนี้ไม่ใช้การจัดอันดับเวลาทดลองตั้งแต่เริ่มจนจบทั้งหมด

4.7 ข้อมูลเฉพาะวิธีและผลบนกรณีร่วม

4.7.1 การใช้ token

ตารางที่ 21 ตารางการใช้ token

รายการ	ChatGPT	GitHub Copilot
กรณีที่มีข้อมูล token	854	854
Token รวม	9,759,806	22,888,274
Token เฉลี่ยต่อกรณี	11,428.34	26,801.26
เวลา P02 เฉลี่ย	36.94 วินาที	174.46 วินาที

เป้าหมาย token คือเฉลี่ยไม่เกิน 13,000 ต่อกรณี โดยไม่มี hard output cap ChatGPT มีค่าเฉลี่ยต่ำกว่าเป้าหมาย ส่วน GitHub Copilot มีค่าเฉลี่ยสูงกว่าเป้าหมาย

Token เป็นข้อมูลเฉพาะบริการ AI ไม่ใช่แทนเวลาหรือหน่วยความจำของเครื่องมืออื่น

4.7.2 การเปรียบเทียบบนกรณีร่วม

ตารางที่ 22 ตารางเปรียบเทียบบนกรณีร่วม

ตัวชี้วัด	กรณีร่วม	ChatGPT	GitHub Copilot
ตรวจพบข้อบกพร่อง	193	154 หรือ 79.79%	146 หรือ 75.65%
Line coverage เฉลี่ย	192	52.23%	48.83%
Condition coverage เฉลี่ย	191	44.96%	42.87%

เลือกเฉพาะ Project-Bug ID ที่ทั้งสองวิธีมีสถานะ DONE และมีค่าของตัวชี้วัดนั้นจริง ผลแสดงว่า ChatGPT มีค่าสูงกว่า GitHub Copilot บนกลุ่มที่ใช้คำนวณ

ผลนี้จำกัดอยู่ในกรณีที่ทั้งสองวิธีประเมินสำเร็จ ไม่ครอบคลุมกรณีที่วิธีใดวิธีหนึ่งไม่สำเร็จ และเป็นการเปรียบเทียบเชิงพรรณนาโดยไม่มีข้อสรุปนัยสำคัญทางสถิติ

4.8 กรณีศึกษาและการตรวจสอบหลักฐาน

ตารางที่ 23 ตารางรายละเอียดกรณีศึกษาที่พบ

กรณีศึกษา	ผลและความหมาย
Symflower ใน Lang-1	สร้าง 12 tests สำหรับ IEEE754rUtils จาก fixed version และผ่านทั้งสองรุ่น แต่ไม่ได้ครอบคลุม NumberUtils ซึ่งเกี่ยวข้องกับ LANG-747
Bounded symbolic hex	ตัวสร้างเฉพาะทางของกลุ่มใน Lang-1 มี 24 tests ล้มเหลวบน buggy 6 รายการและ fixed 0 รายการ โดยมี Line coverage 54/380
False positive ของ Symflower	Lang-19, Lang-20, Lang-22 และ Lang-36 ถูกแก้ผลหลังพบ developer trigger tests ที่ parser ตัดออกไม่ครบ
Coverage recovery	ChatGPT ใน Jsoup-4/9 และ Copilot ใน Jsoup-4/6 มีผลทดสอบ แต่ instrumentation ของ Entities.class ล้มเหลว จึงไม่มีค่า coverage
Source verification	ChatGPT ใน Jsoup-71 มี hash ของ P04 validation แตกต่างจาก final source แต่ final source, archive และ final result ตรงกัน

สำหรับ Symflower ใน Lang-1 เมื่อวัดเฉพาะ IEEE754rUtils ได้ Line coverage 28/57 หรือ 49.1% และ Condition coverage 20/40 หรือ 50.0% แต่ coverage ของ NumberUtils เป็นศูนย์ การทดลองเพิ่มเติมกับ NumberUtils.createNumber ใน workspace ขนาดเล็กสร้างชุดทดสอบไม่สำเร็จ โดยมีข้อความเกี่ยวกับ unknown function, unknown import และ unsupported type

Bounded symbolic hex เป็นการทดลองเฉพาะทางแยกจาก Symflower จึงไม่รวมผลตรวจพบดังกล่าวเป็นผล Symflower ในตาราง 854 กรณี

กรณี coverage recovery พบข้อความ Method code too large ระหว่าง instrumentation ผล fault detection จึงจับเก็บได้แต่ไม่มี coverage และไม่แทนค่าที่ขาดด้วยศูนย์

สำหรับ Jsoup-71 logs ยืนยันว่า fixed version ผ่านและ buggy version มี tests ล้มเหลว 7 รายการ จึงบันทึก final-source verification เพิ่มเติมโดยคง P04 validation เดิมไว้ สาเหตุของ hash ที่ต่างกันยังไม่มีข้อยืนยัน

บทที่ 5

อภิปรายผล และสรุปผล

5.1 อภิปรายความสำเร็จของกระบวนการ

ผลการทดลองแสดงว่าการสร้างไฟล์ทดสอบ การคอมไพล์ การรัน และการผ่าน validation เป็นคนละขั้นตอน การสร้าง source ได้จึงไม่เพียงพอสำหรับยืนยันว่าชุดทดสอบพร้อมใช้งาน

NSGA-II มีผลประเมินจำนวนมาก แต่สถานะ ok รวมทั้งชุดทดสอบที่ผ่านและล้มเหลวบน fixed version ส่วน AI ใช้การผ่าน fixed validation เป็นเงื่อนไขของ DONE สำหรับ Symflower จำนวนไฟล์และ declarations แสดงผลของการสร้าง source แต่ไม่ได้ยืนยันความถูกต้องและการทำงานของทุกชุด

ความแตกต่างของสถานะเหล่านี้ทำให้จำนวนผลสำเร็จไม่สามารถใช้แทนคุณภาพประเภทเดียวกันทั้งหมด อย่างไรก็ตาม การแยกขั้นตอนช่วยแสดงตำแหน่งที่แต่ละวิธีติดข้อจำกัดได้ชัดเจน

5.2 อภิปรายการตรวจพบข้อบกพร่องและ test oracle

เมื่อเปรียบเทียบสองวิธี AI ที่ใช้เกณฑ์ตรวจพบ fault เหมือนกัน ChatGPT มีจำนวนกรณีที่ตรวจพบมากกว่า ส่วน GitHub Copilot มีอัตราตรวจพบในกลุ่ม DONE ใกล้เคียงกัน แต่มีจำนวนกรณี DONE น้อยกว่า

อัตราต่อกรณีทั้งหมดสะท้อนผลที่กระบวนการทำได้บนขอบเขตการทดลอง ส่วนอัตราเฉพาะกลุ่มที่มีผลสะท้อนผลหลังผ่านขั้นตอนก่อนหน้านี้ กลุ่มที่ประเมินสำเร็จอาจมีลักษณะข้อบกพร่องต่างจากกลุ่มที่ไม่สำเร็จ จึงไม่สรุปคุณภาพจากเปอร์เซ็นต์ประเภทเดียว

NSGA-II ใช้การผ่าน buggy และล้มเหลวบน fixed เพื่อแสดงความต่างของพฤติกรรม ขณะที่ AI ใช้ fixed validation ตรวจ expected behavior ก่อนประเมิน buggy version ผลทั้งสองประเภทจึงมีความสัมพันธ์กับ test oracle ต่างกัน

สำหรับ Symflower ผลตรวจพบเป็นศูนย์เกิดภายใต้ขอบเขตและ configuration ที่ทดลอง ไม่ยืนยันว่า Symbolic Execution ไม่สามารถตรวจพบข้อบกพร่องทุกประเภท ข้อจำกัดที่พบครอบคลุมการวิเคราะห์ source การสร้างโค้ดที่เข้ากันได้ และการสร้าง assertions ที่แยกพฤติกรรมของโปรแกรมสองรุ่น

ข้อมูลข้อบกพร่องที่ส่งให้ AI อาจช่วยให้สร้าง input และ expected values ที่เกี่ยวข้องกับพฤติกรรมที่ต้องการ แต่การทดลองนี้ไม่ได้แยกปัจจัยดังกล่าวออกมาทดสอบโดยเฉพาะ จึงไม่ยืนยันว่าเป็นสาเหตุหลักของความแตกต่างทั้งหมด

5.3 อภิปรายความครอบคลุมของโค้ด

Coverage แสดงส่วนของโปรแกรมที่ถูกเรียกใช้งาน แต่ไม่ยืนยันว่าชุดทดสอบตรวจสอบผลลัพธ์ได้ถูกต้อง ชุดทดสอบที่ครอบคลุมโค้ดมากอาจไม่ตรวจพบข้อบกพร่อง ขณะที่ชุดทดสอบขนาดเล็กอาจตรวจพบได้หาก input และ assertions ตรงกับเงื่อนไขที่ผิด

ค่าเฉลี่ย coverage ของแต่ละวิธีคำนวณจากกลุ่มกรณีต่างกัน จึงอาจได้รับอิทธิพลจากความยากและองค์ประกอบของกลุ่ม การเปรียบเทียบบนกรณีร่วมลดความต่างด้านชุดข้อมูลได้บางส่วน แต่ยังจำกัดอยู่ในกรณีที่ทั้งสองวิธีมีผลพร้อมกัน

5.4 อภิปรายเวลาและทรัพยากร

NSGA-II รอบที่ใช้ search budget 60 วินาทีมีเวลา generation เฉลี่ยต่ำกว่ารอบ 150 วินาที แต่มีจำนวนกรณีที่สร้างชุดทดสอบไม่สำเร็จมากกว่า ความแตกต่างนี้เกิดรวมกับการใช้สองเครื่องในรอบที่ 2 และความแปรผันจากการรันซ้ำ จึงไม่อธิบายว่าเกิดจาก budget เพียงอย่างเดียว

Symflower มีมัลฐานเวลา generation ต่ำกว่าค่าเฉลี่ยมาก แสดงว่าบางกรณีใช้เวลามากกว่ากรณีส่วนใหญ่ ส่วนบริการ AI มีเวลาและ token ต่างกันแม้ใช้ prompt templates ร่วมกัน

เวลาที่รายงานมีขอบเขตและฐานการคำนวณต่างกัน จึงยังไม่ใช่การเปรียบเทียบเวลารวมของทุกขั้นตอน Token และ search budget ช่วยอธิบายทรัพยากรของแต่ละวิธี แต่ไม่ใช่หน่วยเดียวกันสำหรับเปรียบเทียบทั้ง 4 วิธี

5.5 ปัญหาและสิ่งที่เรียนรู้

ปัญหาที่พบครอบคลุมการเตรียมโครงการ dependencies การเลือกคลาสเป้าหมาย การสร้างโค้ดทดสอบที่เข้ากันได้ และข้อจำกัดของเครื่องมือประเมิน

การแก้ false positive ของ Symflower แสดงว่าการอ่าน metadata และการแยก developer tests มีผลต่อข้อสรุป ส่วนการตรวจ source hash และ archive ช่วยยืนยันว่าผลประเมินเกิดจากชุดทดสอบที่จัดเก็บจริง

กรณี instrumentation ล้มเหลวแสดงว่าผลทดสอบและ coverage อาจมีความพร้อมต่างกัน การแยกสถานะของตัวชี้วัดและไม่แทนข้อมูลที่ไม่มีค่าด้วยศูนย์จึงช่วยรักษาความหมายของข้อมูล

การเปลี่ยน protocol, configuration และกติกาการรวมผลมีผลต่อความดีความ การเก็บข้อมูลชุดการรันและหลักฐานรายการจึงเป็นส่วนสำคัญของการประเมิน ไม่ใช่เพียงข้อมูลประกอบท้ายรายงาน

5.6 ข้อจำกัดของการศึกษา

ตารางที่ 24 ตารางรายละเอียดข้อจำกัดที่พบ

ประเด็น	ข้อจำกัด
นิยามผลสำเร็จ	สถานะของแต่ละวิธีมีความหมายต่างกัน
Fault detection	ผล NSGA-II ที่รายงานเป็น fault มีทิศทางผ่านบน buggy และล้มเหลวนบน fixed ซึ่งต่างจากเกณฑ์ผ่านบน fixed และล้มเหลวนบน buggy ที่ใช้กับ AI
Coverage	ขอบเขตการวัดต่างกัน โดย Symflower ในข้อมูลหลักรวม developer tests
กลุ่มข้อมูล	ค่าเฉลี่ยและอัตราเฉพาะกลุ่มใช้กรณีที่มีผลต่างกัน
เวลา	ขั้นตอนที่ครอบคลุมและฐานการคำนวณต่างกัน
การรันซ้ำ	NSGA-II มีความแปรผันและใช้กติกาเลือกผลล่าสุดที่สำเร็จ
Protocol	AI ไม่ได้ใช้ revision เดียวกันตลอดทุกกรณี

ตารางที่ 24 ตารางรายละเอียดข้อจำกัดที่พบ (ต่อ)

จำนวนการทดสอบ	Declarations ไม่ใช่ executions และไม่มีจำนวน executions ครบทุกกรณี
การวิเคราะห์	ผลเป็นสถิติเชิงพรรณนา ไม่มีข้อสรุปนัยสำคัญทางสถิติ

ผลการศึกษาคครอบคลุมชุดข้อมูลและเงื่อนไขที่ทดลอง ไม่ใช่ยืนยันประสิทธิภาพกับโปรแกรม Java ทุกประเภท

5.7 สรุปผลการศึกษา

โครงการทดลองสร้างชุดทดสอบด้วย NSGA-II, Symflower, ChatGPT และ GitHub Copilot บนข้อบกพร่องจาก Defects4J จำนวน 854 กรณี

NSGA-II มีผลประเมิน 789 และ 669 กรณีในสองรอบ และแยกพฤติกรรมของโปรแกรมสองรุ่นได้ 111 และ 95 กรณี ตามทิศทางที่ใช้ Symflower มีไฟล์ทดสอบใน 156 กรณี รวม 10,677 declarations แต่ไม่มีผลตรวจพบในข้อมูลหลักหลังแก้ false positive ส่วน ChatGPT และ GitHub Copilot มีชุดทดสอบที่ผ่าน fixed validation และประเมินสำเร็จ 561 และ 229 กรณี ตรวจพบข้อบกพร่อง 403 และ 166 กรณีตามลำดับ

ผลแสดงว่าความสามารถในการสร้างชุดทดสอบ ความพร้อมในการใช้งาน การตรวจพบข้อบกพร่อง coverage และทรัพยากรเป็นคุณสมบัติที่ต้องพิจารณาร่วมกัน การใช้ตัวชี้วัดเดียวไม่เพียงพอสำหรับอธิบายคุณภาพของกระบวนการทั้งหมด และการเปรียบเทียบมีความหมายภายใต้ नियาม ขอบเขต และหลักฐานที่ระบุไว้